TP5 - Sensors

Mobile & Ubiquitous Computing 2026/2027
Ciências, Departamento de Informática

Diogo Branco

Sensors

#5 Class

Evaluation

Assignment #4 - Persistent Data

0



Sensors

Sensors Examples

- Camera
- Accelerometer
- Location (GPS/Wifi)
- Bluetooth
- Luminosity
- Touch
- Others

https://developer.android.com/guide/topics/sensors

Sensors Tutorials

- Location (GPS/Wifi)
- Gesture
- Camera

1



Location

Location

1. Create a new project empty activity
2. Add permissions request to the manifest (next slide)
3. Check and request permissions in runtime
4. Get last location

Manifest

You declare a need for foreground location when your app requests either the `ACCESS_COARSE_LOCATION` permission or the `ACCESS_FINE_LOCATION` permission, as shown in the following snippet:

```
<manifest ... >
  <!-- Always include this permission -->
  <uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />

  <!-- Include only if your app benefits from precise location access. -->
  <uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
</manifest>
```

To learn more about permissions*

https://developer.android.com/develop/sensors-and-location/location/permissions

Create Location Services Client

In your activity's `onCreate()` method, create an instance of the Fused Location Provider Client as the following code snippet shows.

Kotlin Java

```kotlin
private lateinit var fusedLocationClient: FusedLocationProviderClient

override fun onCreate(savedInstanceState: Bundle?) {
    // ...

    fusedLocationClient = LocationServices.getFusedLocationProviderClient(this)
}
```

To learn more about location* https://developer.android.com/develop/sensors-and-location/location/retrieve-current

Permission Request

```
private lateinit var fusedLocationClient: FusedLocationProviderClient
val locationPermissionRequest = registerForActivityResult(
ActivityResultContracts.RequestMultiplePermissions()
) { permissions ->   when {
permissions.getOrDefault(Manifest.permission.ACCESS_FINE_LOCATION,
false) -> {
            // Precise location access granted.
        }
permissions.getOrDefault(Manifest.permission.ACCESS_COARSE_LOCATION,
false) -> {
            // Only approximate location access granted.
        }
        else -> {
            // No location access granted.
        }
    }
}
```

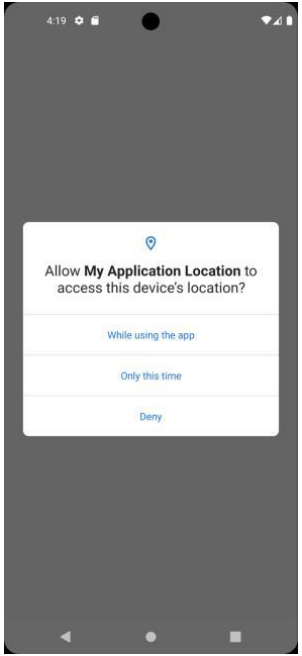
Permission Check/Request onCreate

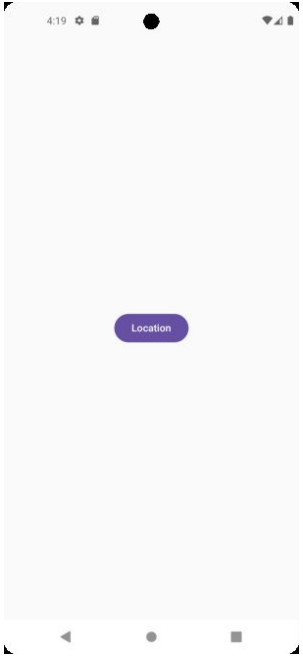
```
if (ActivityCompat.checkSelfPermission(
        this,
        Manifest.permission.ACCESS_FINE_LOCATION
    ) != PackageManager.PERMISSION_GRANTED &&
ActivityCompat.checkSelfPermission(
        this,
        Manifest.permission.ACCESS_COARSE_LOCATION
    ) != PackageManager.PERMISSION_GRANTED
) {
    locationPermissionRequest.launch(
        arrayOf(
            Manifest.permission.ACCESS_FINE_LOCATION,
            Manifest.permission.ACCESS_COARSE_LOCATION
        )
    )
    return
}
```

Add a button to request last known location (inside activity)

```
@SuppressLint("MissingPermission")
fun getLastLocation() {
   Log.d("TAG", "Teste")
   fusedLocationClient.lastLocation
       .addOnSuccessListener { location: Location? ->
           Log.d("Teste", "Location:" + location.toString())
       }
}
@Composable
fun LocationApp(modifier: Modifier = Modifier  ) {
   Column(  modifier = Modifier.padding(30.dp).then(Modifier.fillMaxSize())
       .then(Modifier.wrapContentSize(Alignment.Center))) {
       Button(onClick = { getLastLocation() }) {
           Text("Location")
       }
   }
}
```

Run the app






```
2024-10-29 16:19:08.382  8943-8967  OpenGLRenderer      com.example.myapplicationlocation  I  Davey! duration=1080ms; Flags=1, IntendedVsync=2021163088698, Vsync=2021330355358, OldestInputEvent=922337
2024-10-29 16:19:08.460  8943-8943  Choreographer       com.example.myapplicationlocation  I  Skipped 54 frames!  The application may be doing too much work on its main thread.
2024-10-29 16:19:12.524  8943-8976  ProfileInstaller    com.example.myapplicationlocation  D  Installing profile for com.example.myapplicationlocation
2024-10-29 16:20:02.324  8943-8943  TAG                 com.example.myapplicationlocation  D  Teste
2024-10-29 16:20:02.345  8943-8943  Teste               com.example.myapplicationlocation  D  Location:Location[fused 37.422094,-122.083922 hAcc=600 et=+31m55s902ms vAcc=??? sAcc=??? bAcc=???]
```

Google Maps + Location (Optional - Recommended)

Requires creating a billing account to access the trial (you should have it from the previous class).

You will not be billed, but follow the same approach if needed.

Add a map to your Android app (Kotlin) – On a new project downloaded from:
https://developers.google.com/codelabs/maps-platform/maps-platform-101-android#0

Learn more about location services:
https://codelabs.developers.google.com/codelabs/while-in-use-location?hl=en#0

2



Gestures

Gestures

Now make the location button draggable

https://developer.android.com/jetpack/compose/touch-input/pointer-input/drag-swipe-fling

**to learn more about gestures follow the link*

https://developer.android.com/develop/ui/compose/touch-input/pointer-input/understand-gestures

3



Camera

Camera

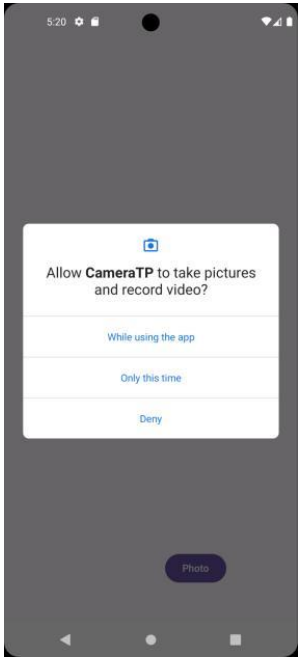
1. Check tutorial on: https://proandroiddev.com/lets-build-an-android-camera-app-camerax-compose-9ea47356aa80

2. Website does not show all the code, but video of the full tutorial: https://www.youtube.com/watch?v=kNx2LuJ_bpM

3. Full working version: https://github.com/tdcolvin/CameraXWorkshop

Camera









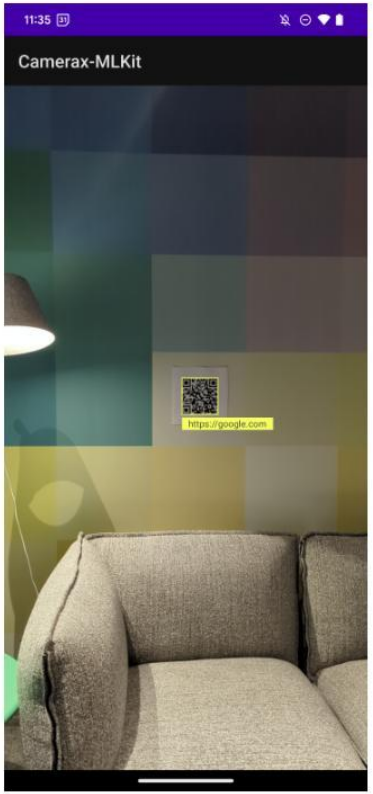
For more on camera:
https://developer.android.com/media/camera/get-started-with-camera

Other resources

ML kit Analyser & Sample - learn how to leverage the camera

https://developer.android.com/media/camera/camerax/mlkitanalyzer

Screenshots



3

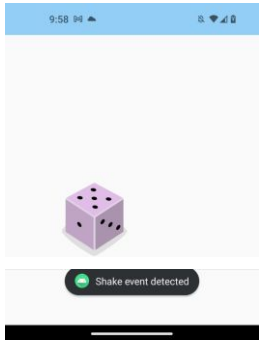


Assignment

Assignment #5 - Drag, Shake & Camera

Using the second assignment Dice Results

1. Roll the dice using the accelerometer (shaking)
2. Make the “Roll” button Draggable
3. As an alternative to shaking the device, users can perform a rapid flick gesture across the screen to throw the dice in a specific direction



Using the 3/4 assignment New API / Remote Storage to allow add and update photos of Rick And Morty locations

1. Add a new screen for camera preview and capture
2. Display the camera capture and location within the app on the current Rick and Morty Location
3. Store the photo taken (local or remote) together with location (latitude and longitude)

